```
// index.php
$dataObj = new stdClass();
$dataObj->name = "A basic object.";

print $twig->render('structured-data.html.twig', [
    'array_data' => [
        'cat' => 'Jess',
        'humans' => ['Anna', 'Paul'],
    ],
    'object' => $dataObj,
]);
```

```
{# views/structured-data.html.twig #}
<div>{{ array_data.cat }}</div>
<div>{{ array_data.humans.0 }}, {{ array_data.humans.1 }}</div>
<div>{{ object.name }}</div>
```

We've introduced two new Twig concepts here: *sequences* and *mappings*. These are the Twig abstractions representing arrays and associative arrays or objects, respectively. Twig dot operator is used to access the attributes of iterables and objects, where attributes are PHP array keys and indices (`$array['humans'][0]` translates to `array.humans.0`, etc.) or object properties (`$object->name` translates to `object.name`). Twig also allows attribute/property access with square bracket notation, for example, `{{ myMapping['foo'] }}`, `{{ mySequence[0] }}`. The template above renders the following output:

```
$ curl http://localhost:8080/index.php
<div>Jess</div>
<div>Anna, Paul</div>
<div>A basic object.</div>
```

Twig Escaping

Part of the reason why we looked at DOM manipulation first is because I wanted you to get to this section of the book already having a firm grasp of the essentials of HTML. This way the focus can be on templating without being cluttered with HTML-101 lessons. Another supremely important takeaway from this section will be escaping any HTML in the data that you use for rendering. We actually touched on this briefly during our